

How can embedding models bind concepts?

Anonymous Authors¹

Abstract

Humans easily determine which color belongs to which shape in multi-object scenes, an ability known as concept binding. Vision-language embedding models such as CLIP struggle with binding: they recognize individual concepts but fail to represent which concepts form which objects. Although CLIP behaves like a bag-of-concepts model in cross-modal retrieval, object information is recoverable from its image and text embeddings separately. We study this tension through the binding function, which maps concepts to scene embeddings. We find that scene embeddings decompose additively into object representations, explaining why uni-modal probes can recover object information. However, CLIP’s binding function is high-complexity, which likely prevents the image and text encoders from learning a shared binding mechanism that generalizes to unseen concept combinations. We then ask whether this limitation is fundamental. We show that it is not. In controlled transformer models trained from scratch, binding generalization emerges with sufficient data coverage. These models learn low-complexity binding functions characterized by multiplicative interactions between concepts, enabling systematic generalization.

1. Introduction

Given a picture of a scene with two objects of different colors and shapes, humans can easily determine which color is associated with which shape, an ability often referred to as concept binding (Treisman & Gelade, 1980; Treisman, 1998). While straightforward for humans, this has proved challenging for modern vision-language embedding systems such as CLIP (Radford et al., 2021). These models often behave like bag-of-concepts systems: they reliably recognize individual concepts (e.g., *red*, *square*), yet struggle to cor-

rectly represent their combinations in multi-object scenes (e.g., distinguishing *red square* vs. *blue square*) (Yuksekgonul et al., 2023; Lewis et al., 2024; Tang et al., 2023). At the same time, within individual modalities of vision and language, object-related information can often be recovered from embeddings using probes (Koishigarina et al., 2025). In other words, despite their failures in multi-object settings, these models appear to internally encode object information to some extent. This tension motivates a closer examination of the geometry of CLIP’s binding: if object information is present, how is it represented, and why does it fail to align across modalities?

We begin by analyzing how multi-object scenes are represented in CLIP. We find that scene embeddings exhibit a clear additive structure: they decompose into sums of object representations. This structure explains why uni-modal probes can recover object information—object-level components are explicitly present in the embedding space and can be manipulated directly. However, this geometric structure alone does not explain binding. We show that CLIP’s binding function, which maps concepts to object representations, is high-complexity and combination-specific. The mapping from concepts to objects differs across concept combinations, effectively requiring memorization of object identities. This prevents generalization to unseen objects and makes it difficult for the image and text encoders to learn a shared binding mechanism, despite exhibiting uni-modal binding.

We then ask whether this limitation is fundamental. We show that it is not. In controlled transformer-based dual-encoder models trained from scratch on synthetic multi-object data, binding generalization emerges with sufficient data coverage. These models learn low-complexity binding functions that reuse structure across concept combinations and generalize to unseen objects. Finally, we analyze the structure of these generalizing binding functions. We find that models that generalize implement multiplicative interactions between concepts, rather than purely additive composition. This multiplicative structure enables generalization and explains how different modalities can produce aligned object representations for unseen concept combinations.

Together, these results show that binding failures in CLIP arise not from the absence of object structure, but from the complexity of the binding function it learns. Embedding

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

models are capable of generalizable binding, but doing so requires learning a low-complexity, systematic concept-to-object mapping that can be shared across modalities.

2. Related work

Binding. The binding problem asks how models associate features that belong to the same object (Greff et al., 2020; Feng & Steinhardt, 2024). Surveys and empirical studies examine binding limits and emergent symbolic mechanisms (Campbell et al., 2024; Assouel et al., 2025). In autoregressive language models, mechanistic studies have identified position-independent binding IDs that link related tokens (Feng & Steinhardt, 2024; Feng et al., 2024). Binding at patch-level of DINO and CLIP has also been studied (Li et al., 2026), showing that vision models naturally develop an ability to bind at a patch level. However, such mechanisms operate at the token level and it remains unclear how embedding models that produce a single vector per input could implement analogous structure. In contrast, we study how binding is realized in the geometry of embedding models: we ask how object representations are constructed from concepts, and argue that the complexity of this mapping explains cross-modal binding failures.

Binding in embedding models. CLIP has been observed to exhibit uni-modal binding (Koishigarina et al., 2025) but fail cross-modally (Lewis et al., 2024; Tang et al., 2023). These failures have been attributed to encoder-level weaknesses such as difficulty with fine-grained details, negations, and spatial reasoning (Tong et al., 2024b;a; Kamath et al., 2023; Li et al., 2025) and over-reliance on concept-level information (Huang et al., 2025). Several works propose fine-tuning approaches to improve CLIP’s compositional understanding, including word order sensitivity and attribute-object association (Yuksekgonul et al., 2023; Ma et al., 2023; Hsieh et al., 2024) and argue of the importance of data (Gurung et al., 2025). Some works suggest that concept recognition and object recognition (binding) are fundamentally at odds, suggesting a trade-off (Kang et al., 2025). In contrast, we show that both concept and object recognition can coexist in the same embedding space, explain how uni-modal binding can succeed via linear probes, and argue that cross-modal binding failures stem from the complexity of how concepts are combined into objects, not from an inherent incompatibility.

Compositional generalization. Research on compositional generalization investigates how models can generalize to novel combinations of known parts. Many approaches attempt to inject this ability into neural networks (Wang et al., 2024; Mahajan et al., 2025; Jarvis et al., 2024), while others study conditions under which it can be achieved (Lippl & Stachenfeld, 2025; Montero et al., 2021; Dittadi et al., 2021; Liang et al., 2025; Schott et al., 2022; Kempf et al., 2025; Okawa et al., 2023). However, this literature largely focuses on single-object settings with factorized concept spaces.

For CLIP specifically, generalization of concept recognition to novel combinations has been studied (Uselis et al., 2025b), but generalizing binding, object-level recognition, to novel combinations remains difficult (Lewis et al., 2024). In contrast, we study multi-object scenes and find that generalization of binding is possible and emerges with data scale: at small scales, object recognition fails to generalize (consistent with prior findings), but concept recognition of novel objects remains robust; at larger scales, both concept and object recognition generalize.

Geometry of learned representations. Many works study the shape of learned features. General notions of the Linear Representation Hypothesis mostly report linearity in language models (Jiang et al., 2024; Park et al., 2023). Abasi et al. (2024) find evidence of disentanglement in CLIP models, Trager et al. (2023) show additive decomposition in CLIP text encoder, and Uselis et al. (2025a); Berasi et al. (2025) show it in vision encoder. However, these analyses typically characterize geometry in terms of single objects. In contrast, we extend this to multi-object scenes and show that scene embeddings admit a two-level additive decomposition into objects and concepts.

3. Formalisation of binding

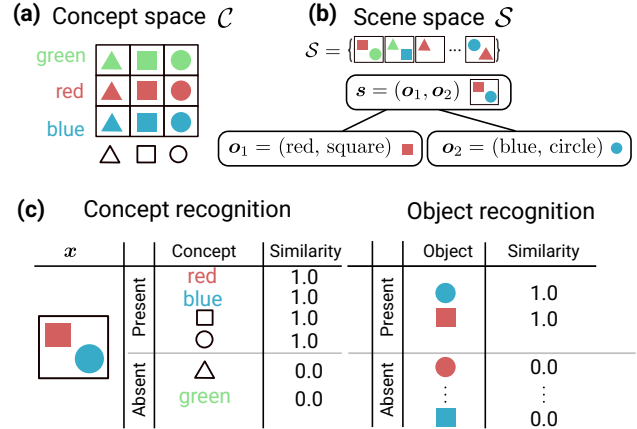


Figure 1. Schematic of the binding setup. (a) Example concept space $\mathcal{C}$ (e.g., color $\times$ shape), where each object is a tuple of concept values. (b) Example scene space $\mathcal{S}$, where a scene s is a tuple of objects (here, two objects (o_1, o_2)). (c) Example of the two recognition criteria: *concept recognition* ranks present concept values above absent ones (Def. 3.3), while *object recognition* ranks present objects above absent concept combinations (Def. 3.4).

Before analyzing how embedding models represent multi-object scenes, we formalize the problem. We define what it means to recognize concepts, recognize objects, and bind them together. This formalization guides §4 and §5.

We assume images and captions depict scenes. A scene contains one or more objects. Each object is specified by values of a fixed set of concepts (e.g. color and shape). We follow prior work (Okawa et al., 2023) in defining objects as combinations of concept values. We extend this formalisation to

scenes containing multiple objects.

Definition 3.1 (Concept space). Let the C concepts be indexed by $i \in \{1, \dots, C\}$, where concept i takes values in a set $\mathcal{C}_i$ (often $|\mathcal{C}_i| = V$ for all i). The *concept space* is the Cartesian product $\mathcal{C} = \mathcal{C}_1 \times \dots \times \mathcal{C}_C$. An *object* is a vector $\mathbf{o} = (c_1, \dots, c_C) \in \mathcal{C}$, where $c_i \in \mathcal{C}_i$ specifies the value for concept i . An object therefore corresponds to a particular combination of concept values.

Definition 3.2 (Scene space). Let $O_{\max} \in \mathbb{N}$ denote the maximum number of objects in a scene. For each $m \in \{1, \dots, O_{\max}\}$, let $\mathcal{C}^m$ be the set of ordered m -tuples of objects. The *scene space* is $\bigcup_{m=1}^{O_{\max}} \mathcal{C}^m$. A *scene* is a tuple $\mathbf{s} = (\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_m) \in \mathcal{S}$, describing a collection of m objects. We assume that each scene $\mathbf{s}$ corresponds to some datapoint $\mathbf{x}_s \in \mathcal{X}$ (e.g. an image).

Models. We study embedding models that encode inputs and queries independently. An input corresponds to a scene, such as an image, while a query specifies the information to retrieve, such as a concept value or an object (e.g. CLIP).

We denote the input encoder by $f : \mathcal{X} \rightarrow \mathbb{R}^d$ and the query encoder by $q : \mathcal{Y} \rightarrow \mathbb{R}^d$. Given an input $\mathbf{x}_s$ and a query $\mathbf{y} \in \mathcal{Y}$, the matching score is given by cosine similarity, $s(\mathbf{x}_s, \mathbf{y}) = \frac{f(\mathbf{x}_s)^\top q(\mathbf{y})}{\|f(\mathbf{x}_s)\| \|q(\mathbf{y})\|}$. We refer to the pair (f, q) as a model.

For a scene $\mathbf{s} = (\mathbf{o}^{(1)}, \dots, \mathbf{o}^{(m)}) \in \mathcal{S}$, we define the set of values taken by concept i across all objects in the scene as $V_i(\mathbf{s}) := \{c_i^{(1)}, \dots, c_i^{(m)}\} \subseteq \mathcal{C}_i$. We also define the set of objects present in the scene as $O(\mathbf{s}) := \{\mathbf{o}^{(1)}, \dots, \mathbf{o}^{(m)}\} \subseteq \mathcal{C}$.

Binding ability. A model exhibits binding ability when it can: (1) *concept-level recognition*: recognize which concept values are present in a scene, (2) *object-level recognition*: recognize which objects are present in a scene.

Definition 3.3 (Concept recognition). A model (f, q) recognizes concepts if for any scene $\mathbf{s} \in \mathcal{S}$ and any concept $i \in [C]$, the model assigns higher scores to all concept values present in the scene than to any absent ones:

$$\min_{v \in V_i(\mathbf{s})} s(\mathbf{x}_s, \mathbf{y}_{i,v}) > \max_{v' \in \mathcal{C}_i \setminus V_i(\mathbf{s})} s(\mathbf{x}_s, \mathbf{y}_{i,v'}). \quad (1)$$

While concept recognition captures the presence of individual concept values, it does not test whether the model can identify which values belong together. This distinction is formalized by object recognition.

Definition 3.4 (Object recognition). A model (f, q) recognizes objects if for any scene $\mathbf{s} \in \mathcal{S}$, the model assigns higher scores to all objects present in the scene than to any objects not present:

$$\min_{\mathbf{o} \in O(\mathbf{s})} s(\mathbf{x}_s, \mathbf{y}_{\mathbf{o}}) > \max_{\mathbf{o}' \in \mathcal{C} \setminus O(\mathbf{s})} s(\mathbf{x}_s, \mathbf{y}_{\mathbf{o}'}). \quad (2)$$

Concept recognition and object recognition capture complementary aspects of how a model represents scenes. Concept recognition identifies which concept values are present, while object recognition identifies which combinations of concept values form objects. Binding concerns the ability to support both levels in a consistent way.

Definition 3.5 (Binding). A model exhibits binding if it satisfies both *concept recognition* (Def. 3.3) and *object recognition* (Def. 3.4), i.e., it can identify which concept values are present in a scene and which values belong together.

The definitions above characterize binding as a behavioral property. We distinguish *cross-modal binding*, where the model (f, q) directly satisfies concept and object recognition via cosine similarity, from *uni-modal binding*, where a probe must be trained on top of embeddings within a single modality. Prior work has shown that CLIP exhibits uni-modal binding but fails cross-modally (Koishigarina et al., 2025). To understand why, we examine how embeddings are constructed from scenes.

Definition 3.6 (Binding function). For a dual-encoder model (f, q) , we define the *binding functions* $B_{\text{img}} : \mathcal{S} \rightarrow \mathbb{R}^d$ and $B_{\text{txt}} : \mathcal{S} \rightarrow \mathbb{R}^d$ as

$$B_{\text{img}}(\mathbf{s}) := f(\mathbf{x}_s), \quad B_{\text{txt}}(\mathbf{s}) := q(\mathbf{y}_s), \quad (3)$$

mapping scenes to embeddings within each modality.

Intuitively, cross-modal binding requires B_{img} and B_{txt} to produce compatible embeddings for the same scene, including novel ones. If both binding functions have low complexity, they are more likely to produce aligned embeddings. If they are high-complexity, each modality may implement binding in its own way, preventing cross-modal alignment.

In the following sections, we first study the *geometry* of CLIP’s binding functions: how scene embeddings decompose into objects and concepts (§4). We then investigate whether these functions can be approximated from concept indices (§5), and study controlled models where binding does generalize to novel concept combinations.

4. Geometry of CLIP’s binding

CLIP embeddings have been shown to encode binding information within each modality (Koishigarina et al., 2025). This raises a question of how such information is organized in the embedding space. Fig. 2 sheds light on this question. It shows an MDS (Borg & Groenen, 2005) projection of embeddings for two-object scenes. Two-object scene embeddings lie roughly between the embeddings of the corresponding single-object scenes, suggesting that scene embeddings are approximately sums of their constituent object embeddings. Motivated by this geometric structure, we study how multi-object scenes are represented in CLIP. For simplicity, we consider scenes with two objects. Let $\mathbf{s} = (\mathbf{o}_1, \mathbf{o}_2)$ be such a scene, and let $\mathbf{x}_s$ denote the corresponding input.

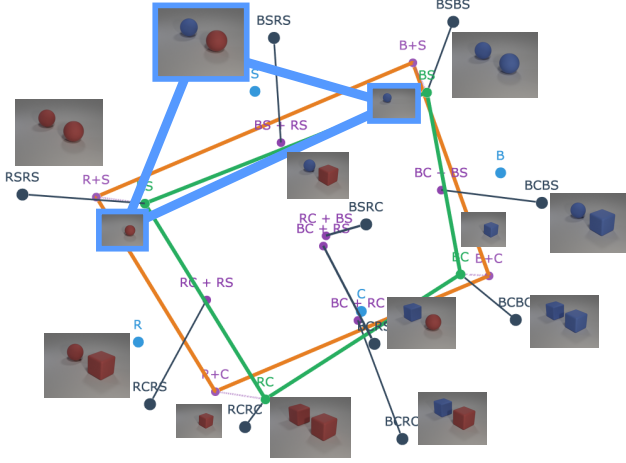


Figure 2. Additive structure in two-object scene embeddings. MDS projection of CLIP embeddings for single- and two-object scenes that vary in color and shape (distances in the plot approximate embedding distances). Labels use R/B for red/blue and C/S for cube/sphere; points correspond to embeddings of the associated single concepts, single-object scenes, and their two-object combinations. Two-object scene embeddings lie near the line segments between the corresponding single-object embeddings; the highlighted triplet (blue outline) illustrates this geometry for one pair of single-object scenes and their combined two-object scene.

We hypothesize that the embedding space exhibits a hierarchical additive structure, where scenes decompose into objects and objects further decompose into concepts:

1. (Level-I) A scene embedding decomposes into a sum of object embeddings.
2. (Level-II) An object embedding decomposes into a sum of concept embeddings and some residual.

Under this hypothesis, the embedding of a two-object scene can be written as follows. Writing each object as a tuple of concept values, $\mathbf{o}_1 = (c_{1,1}, \dots, c_{1,C})$ and $\mathbf{o}_2 = (c_{2,1}, \dots, c_{2,C})$:

$$f(\mathbf{x}_s) = \mathbf{u}_{\mathbf{o}_1} + \mathbf{u}_{\mathbf{o}_2} \approx \sum_{i=1}^C \mathbf{u}_{c_{1,i}} + \sum_{i=1}^C \mathbf{u}_{c_{2,i}}. \quad (4)$$

We empirically evaluate both levels of this decomposition in the following sections.

4.1. Level-I decomposition

We test the first claim of the Level-I decomposition through two metrics: (1) retrieval score, (2) probing accuracy, and (3) R^2 score. Details for the text encoder decomposition are provided in §C.2; experimental results in §C.1.

Results in Tab. 1. The majority of the embedding is accounted by the object embeddings (97% R^2 for CLIP-B/32).

Editing scene embeddings via object replacement. The Level-I decomposition in (4) has a concrete implication. If a scene embedding is well approximated by a sum of object components, then editing a multi-object embedding by re-

Table 1. Summary of decomposition experiment results. Scores for R^2 , retrieval accuracy, and probing accuracy.

Dataset	Model	R^2	Retrieval	Probing
Text	Guess	—	1 / 160K	1 / 400
	Random	0.47 / 0.69	0.42	0.82
	CLIP	0.90 / 0.92	0.97	0.99
PUG:SPARE	Random	0.47 / 0.53	0.25	0.90
	CLIP	0.75 / 0.84	0.93	0.98
CLEVR-2D	CLIP	0.75 / 0.77	0.82	0.99
	DINOv2	0.78 / 0.79	0.68	0.97
CLEVR	CLIP	0.78 / 0.83	0.94	0.96
	DINOv2	0.85 / 0.88	0.86	0.96

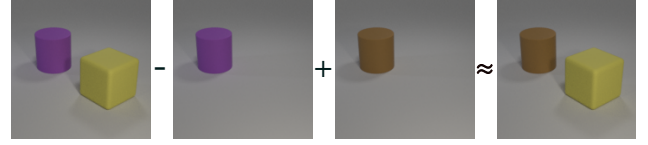


Figure 3. Object-level interventions via embedding arithmetic. An object is removed from a scene embedding and replaced with another object, producing an embedding of the edited scene.

moving one object component and inserting another should produce a meaningful counterfactual representation. We examine this through a object replacements in embeddings.

Setup. Let $\mathbf{s} = (\mathbf{o}_1, \mathbf{o}_2)$ be a two-object scene, and let $\mathbf{o}'_1$ be a counterfactual object obtained by changing one concept value of $\mathbf{o}_1$ (e.g. color). We construct an edited embedding by substituting the object component of $\mathbf{o}_1$ with that of $\mathbf{o}'_1$:

$$\tilde{\mathbf{z}} := f(\mathbf{x}_s) - \mathbf{u}_{\mathbf{o}_1} + \mathbf{u}_{\mathbf{o}'_1}. \quad (5)$$

Object components $\mathbf{u}_{\mathbf{o}}$ are estimated in three ways. We estimate $\mathbf{u}_{\mathbf{o}}$ using dataset averages (with and without position) and single-object scenes as in Fig. 3, and evaluate edited embeddings with retrieval and probing metrics; full details are in §E. We report results for text encoders (synthetic captions; §C.2) and for image encoders on CLEVR, CLEVR-2D, and PUG:SPARE in Tab. 2.

Results. Replacing an object in a multi-object scene embedding with another object produces embeddings that behave like the intended counterfactual scenes under both probing and retrieval. Notably, object components obtained from single-object scenes can be directly reused to edit multi-object scene embeddings. This supports the Level-I decomposition hypothesis that scene embeddings are composed of additive object-level components.

Effect of component removal on object recognition. So far we quantified how well scenes can be reconstructed from objects and concepts in terms of variance explained. Here we make a stronger, task-relevant claim: *decodability* of objects (resp. concepts) relies on the presence of object-level (resp. concept-level) components in the representation.

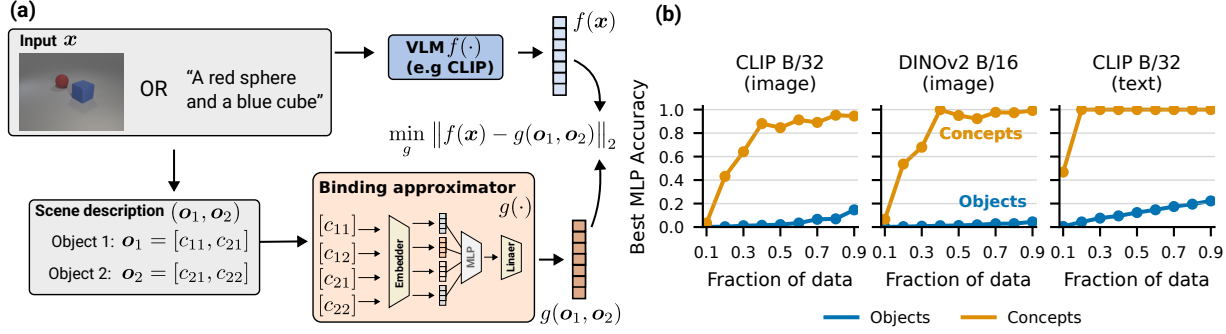


Figure 4. CLIP’s binding function is high-complexity. (a) We train a binding approximator $g(o_1, o_2)$ (a single-layer MLP) to predict CLIP scene embeddings $f(x)$ from concept indices describing the objects present, minimizing (6). (b) Maximum accuracy achieved across all MLP capacities and training coverages for concept vs. object recognition on scenes composed of held-out objects. Predicting concepts is high, confirming that additive structure suffices to capture concept-level information. Object recognition is worse, indicating that predicting how concepts combine into objects requires high model complexity.

Table 2. Object-level decomposition enables controlled edits of scene embeddings. Probing and retrieval performance under object replacement interventions. Each entry reports three values corresponding to object embeddings obtained by scene averaging, position-specific averaging, and single-object scenes, respectively.

Dataset	Model	Probing	Retrieval
Text	CLIP	0.99 / 0.99 / 0.69	0.99 / 0.99 / 0.96
CLEVR	CLIP	0.98 / 0.98 / 0.86	1.00 / 1.00 / 0.97
	DINO	0.97 / 0.98 / 0.77	0.89 / 0.95 / 0.86
CLEVR-2D	CLIP	0.98 / 0.98 / 0.92	0.99 / 0.99 / 0.97
	DINO	0.97 / 0.98 / 0.72	0.97 / 0.97 / 0.88
PUG:SPARE	CLIP	0.94 / 0.95 / –	0.86 / 0.94 / –
	DINO	0.97 / 0.97 / –	0.48 / 0.76 / –

Setup. We perform *targeted ablations* on the scene embedding and then fit linear probes on the residuals. Using the decomposition in (4), we define the *concept part* of a scene as $\sum_i u_{c_{1,i}} + \sum_i u_{c_{2,i}}$, and the *object part* as $u_{o_1} + u_{o_2}$. We then form intervened embeddings by subtracting these components, e.g. $f(x_s) - \sum_i u_{c_{1,i}} - \sum_i u_{c_{2,i}}$ (remove concepts) or $f(x_s) - u_{o_1} - u_{o_2}$ (remove objects), and train probes on the resulting residual embeddings. As a control, we repeat the same ablations using *permuted-ID* concept/object components (details in §D).

Given the evidence of the embedding decomposition into objects, we take the u_o to be *object embeddings*. We analyse their composition in the following section.

4.2. Level-II decomposition

We now examine the decomposition of objects into concepts. A purely additive decomposition corresponds to a bag-of-concepts; while it can explain much of the variance, it cannot encode within-object associations and support binding. We quantify how much variance is explained by (only-)objects vs (only-)concepts using R^2 in Tab. 4.

Results. A bag-of-concepts representation explains a large

Table 3. Decodability results. We report probe accuracy after subtracting components from the embedding (remove concept/s/object) or their permuted-ID controls, for both text and image encoders and their random-init counterparts.

Model	Intervention	Text		Image	
		Conc.	Obj.	Conc.	Obj.
Random	None	0.94	0.60	0.76	0.71
	Subtract concepts	0.02	0.26	0.06	0.66
	Subtract objects	0.01	0.01	0.04	0.00
	Permute concepts	0.87	0.68	0.74	0.72
	Permute objects	0.83	0.93	0.78	0.88
CLIP-B/32	None	1.00	1.00	0.94	0.96
	Subtract concepts	0.06	0.99	0.05	0.85
	Subtract objects	0.05	0.04	0.02	0.01
	Permute concepts	0.92	0.99	0.99	0.97
	Permute objects	0.96	1.0	0.86	0.92

fraction of embedding variance, up to 84% for the text encoder. Representing scenes as bags of objects explains roughly 10% more variance. Thus, most variance is captured by concepts, with object representations contributing additional structure.

Takeaway §4: Scene embeddings contain an explicit, manipulable object-level component: it explains most of the variance, drives object decodability, and supports counterfactual object edits that remain consistent under probing and retrieval.

5. Binding generalization

Having established that scene embeddings decompose into object and concept representations, we next examine whether the binding function from concepts to objects generalizes, quantify its complexity, and analyze its structure.

5.1. CLIP’s binding function does not generalize

Inspired by prequential/MDL perspectives that interpret simplicity as fast generalization from limited data (Blier & Ollivier, 2018; Elmoznino et al., 2025), we measure how

Table 4. Additive decomposition results. We report R^2 for predicting (i) the scene embedding from only object indices (only-objects; cf. Tab. 1) and (ii) the object/scene embeddings from only concept indices (only-concepts).

Model	Scenes from objects R^2	Objects from concepts R^2	Scenes from concepts R^2
Random-embeds	0.01	0.01	-0.03
Random-init (txt)	0.47 / 0.69	0.56 / 0.80	0.39 / 0.56
CLIP-B/32 (txt)	0.90 / 0.92	0.88 / 0.90	0.83 / 0.84
Random-init (img)	0.21 / 0.50	0.25 / 0.57	0.14 / 0.34
CLIP-B/32 (img)	0.37 / 0.83	0.35 / 0.83	0.30 / 0.71

generalization depends on training-set size and MLP capacity (width). Concretely, we study the sample-capacity requirements of a 1-hidden-layer ReLU MLP whose width is varied, trained to approximate a binding map from discrete concept values to scene embeddings.

Setup. We train an approximator $g : \mathcal{C}^2 \rightarrow \mathbb{R}^d$ to predict scene embeddings from concept indices (Fig. 4(a)). For a scene $s = (\mathbf{o}_1, \mathbf{o}_2)$ where each object $\mathbf{o}_i = (c_{i1}, c_{i2})$ is specified by concept values, we minimize

$$\min_g \sum_{s \in \text{train}} \|f(\mathbf{x}_s) - g(\mathbf{o}_1, \mathbf{o}_2)\|^2, \quad (6)$$

where $f(\mathbf{x}_s)$ is the CLIP embedding of scene s . We vary (1) MLP hidden layer width $\{64, 256, 1024, 4096\}$, and (2) the fraction of objects seen during training $\{10\%, \dots, 90\%\}$. At test time, we evaluate on scenes containing only held-out objects. We measure whether the predicted embeddings support both *concept recognition* (Def. 3.3) and *object recognition* (Def. 3.4) by using the linear probes trained on the scene embeddings. See details in §C.3.

Results. Fig. 4(b) reports the best accuracy across the MLPs’ configurations. Concept recognition yields $\geq 80\%$ accuracy among all conditions where the fraction of data exceeds 0.3, whereas object recognition is substantially lower, including at large widths and high training coverage. This gap indicates that correctly predicting object-level structure markedly harder than concept-level structure, consistent with a higher effective complexity of object-level binding under this probe family. Among encoders, the text encoder performs slightly better than the image encoders; CLIP image embeddings outperform DINO but remain far below concept recognition, never exceeding 20% accuracy.

Takeaway §5.1: For CLIP, even high-capacity MLPs fail to generalize to unseen concept combinations. This may explain cross-modal binding failures: a complex, non-generalizing mapping prevents encoders from aligning on novel objects.

5.2. It is possible to have a generalizable binding model

The previous section showed that CLIP’s binding function is hard to approximate and does not generalize to unseen

concept combinations. This raises a natural question: is this failure inherent, or can embedding models learn binding that *does* generalize when the data and training signal are controlled? To answer this, we train CLIP-style dual-encoder transformers on synthetic multi-object scenes where we can precisely vary object-space coverage (Fig. 5).

Setup. We illustrate the process in Fig. 5. We use synthetic scenes with up to 2 objects; each object is defined by C concepts with V values ($|O| = V^C$), and we vary C , V , and the training fraction of seen objects from 0.1 to 0.9. Train scenes draw only from the training split; test scenes are composed entirely of held-out objects. We use two transformer (Vaswani et al., 2023) encoders (query and scene); we refer to them as “text” and “vision” by analogy only since both process tokenized sequences, differing only in whether they encode object/concept queries or full scenes. Both encoders output embeddings in $\mathbb{R}^{512}$ and have $\approx 20\text{M}$ parameters each. We train end-to-end with SGD (AdamW) on a contrastive retrieval objective (Radford et al., 2021); see §B for full details.

Metrics. We evaluate both concept and object recognition. For concepts, we test against all $C \cdot V$ values. For objects, testing against all V^C is infeasible; instead, we use $\text{Swap}(\mathbf{o}_1, \mathbf{o}_2)$, objects formed by exchanging concept values between $\mathbf{o}_1$ and $\mathbf{o}_2$, which directly tests binding by ranking correct object identities against negatives.

Results. We observe two key findings (Fig. 6). First, *compositional generalization emerges*: object recognition on held-out objects starts near the bag-of-concepts baseline but rises sharply with coverage, reaching near-perfect accuracy despite all test objects being unseen. Second, *concepts are easier than objects*: concept recognition generalizes early, while object recognition requires substantially more coverage. When the number of possible objects is small, generalization requires observing more objects, consistent with prior findings in low-combinatorial regimes (Lewis et al., 2024). As the number of possible objects increases, the fraction of objects required for reliable object recognition decreases substantially. For example, when $|O| = 400$, high accuracy is achieved only after observing roughly 50% of objects, whereas for $|O| \geq 2,500$, generalization emerges at around 30% coverage. In the largest setting ($|O| = 125,000$), increasing coverage from 30% to 40% yields a sharp transition from below-chance to near-perfect object recognition. Notably, this occurs despite the corresponding scene space being extremely large (e.g., $(50^3)^2 \approx 15$ billion two-object scenes for $C = 3$, $V = 50$).

Takeaway §5.2: In controlled transformer-based models trained end-to-end with SGD, binding *does* generalize: object recognition can approach perfect accuracy, while concept recognition generalizes much earlier.

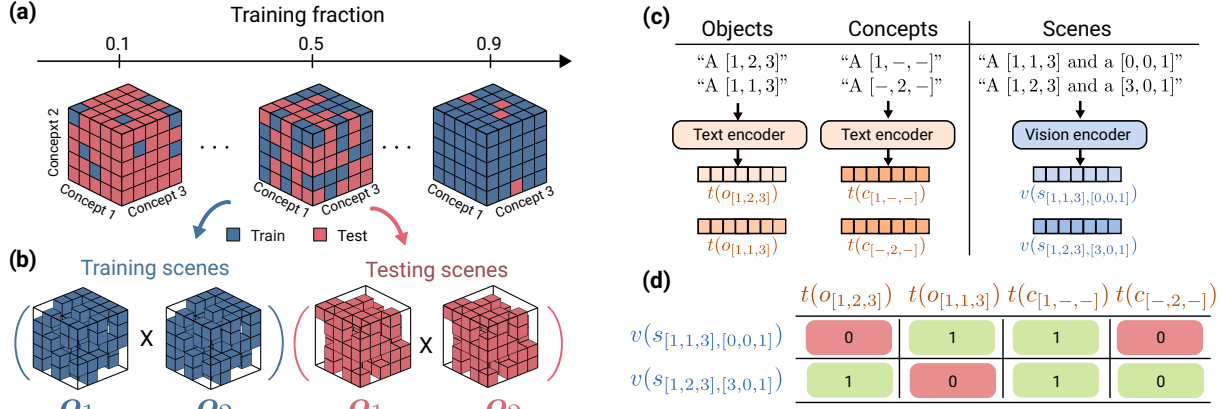


Figure 5. Controlled setup for studying generalizable binding. We train transformer-based embedding models on synthetic multi-object data to test whether binding can generalize to entirely unseen objects. (a) **Data design:** We vary the training coverage ρ_{train} from 0.1 to 0.9, controlling what fraction of the object space the model observes during training. (b) **Scene construction:** Training scenes are composed of objects from the training split (blue); test scenes are composed of objects from the held-out split (red). Crucially, test objects have concept configurations that never appeared during training. (c) **Encoder design:** Following CLIP’s architecture, we use two independent encoders: a “text” encoder that embeds individual objects or concepts, and a “vision” encoder that embeds full scenes. Both produce embeddings in $\mathbb{R}^{512}$. (d) **Training objective:** We optimize a contrastive retrieval loss using cosine similarity. The table shows a training batch where rows are scene embeddings $v(s)$ and columns are object/concept embeddings $t(\cdot)$; green indicates matching pairs, red indicates mismatches.

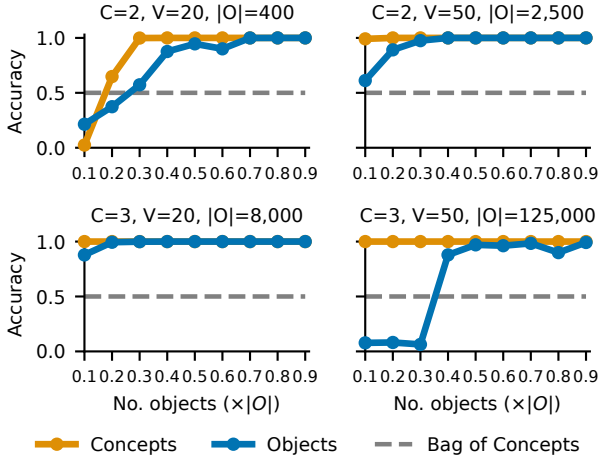


Figure 6. Binding generalization emerges with scale. Test accuracy on held-out objects as a function of training coverage. Each panel varies object complexity (C concepts, V values, $|O| = V^C$ objects). Concept recognition (orange) generalizes readily; object recognition (blue; binding) requires more coverage but reaches high accuracy, surpassing the bag-of-concepts baseline (dashed).

5.3. Low-complexity binding function enables generalization

In §5.1 we operationalized *binding complexity* via generalization of an MLP approximator: even wide MLPs struggled to predict CLIP embeddings on held-out objects, revealing a high-complexity, non-generalizing binding map. Here we apply the same diagnostic to the controlled models from §5.2 and test the converse prediction: models that *do* generalize binding should admit a low-complexity (easy-to-approximate) concept-to-object mapping.

Setup. We measure binding complexity exactly as in §5.1: we train MLPs of varying width to predict scene embeddings from discrete concept indices and evaluate on scenes composed of held-out objects, reporting concept vs. object recognition on the predicted embeddings. If binding is low-complexity, small MLPs should generalize; if it is high-complexity (as in CLIP), generalization should require large capacity and still fail.

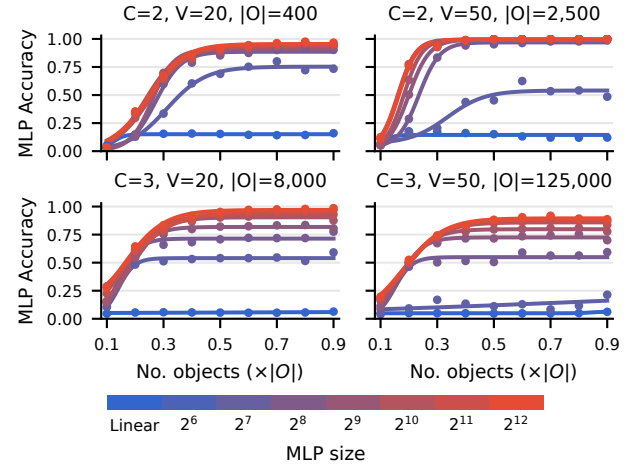


Figure 7. Binding functions can be approximated by low-capacity models. Even single-layer MLPs with small hidden dimensions achieve high accuracy, suggesting that the learned binding operation has low computational complexity. Each panel varies the number of concepts C and concept values V .

Results. Fig. 7 shows that for controlled models that generalize, the binding map is easy to approximate in the sense that even small nonlinear MLPs achieve strong accuracy. At the same time, as the object space grows (larger C and V), the

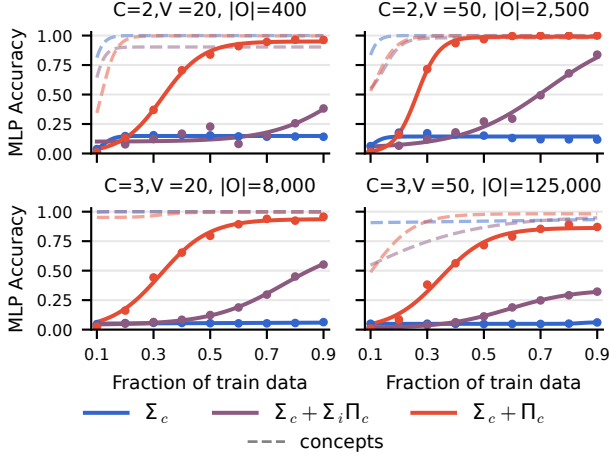


Figure 8. Multiplicative structure best explains binding. Accuracy of different functional forms for predicting scene embeddings from concept indices, evaluated on held-out objects. Dashed: concept recognition; solid: object recognition.

performance gap between the smallest nonlinear MLP and the largest-capacity MLP increases, indicating that the binding map does become moderately more complex with scale. This contrasts with §5.1, where even high-capacity MLPs fail to generalize object recognition on CLIP embeddings.

Takeaway §5.3: In generalizing transformer-based models, binding function is *approximately* low-complexity: even small nonlinear MLPs generalize well.

5.4. Multiplicative structure best explains binding

We next ask what functional form the learned binding takes. We compare simple, structured probes that predict scene embeddings from concept values via additive and multiplicative composition:

1. **Sum of concepts:** $g(s) = \sum_{c \in \mathcal{C}(s)} u_c$.
2. **Sum + per-object products:** $g(s) = \sum_{c \in \mathcal{C}(s)} u_c + \sum_{i=1}^2 \prod_{c \in \mathcal{C}(o_i)} v_{i,c}$.
3. **Sum + multiplication of concepts:** $g(s) = \sum_{c \in \mathcal{C}(s)} u_c + \prod_{i=1}^2 \prod_{c \in \mathcal{C}(o_i)} v_{i,c}$.

We fit each form by learning the parameters $\{u_c\}$ and $\{v_{i,c}\}$ from training scenes, and evaluate on held-out objects via concept and object recognition.

Results. Fig. 8 shows that all probes do well at concept recognition, but the purely additive baseline fails at object recognition. Adding multiplicative interactions recovers binding generalization, with the full product form performing best, especially in larger object spaces.

Multiplicative structure underlies generalization. To test whether multiplicative relationship holds more broadly than on the four considered models, we examine the correlation

between a model’s generalization ability and how well its embeddings are approximated by multiplicative probes. For each of ~ 500 models trained with varying hyperparameters across all object space sizes, we fit the multiplicative probe using 50% of the object space and measure both (1) the model’s object recognition accuracy on held-out objects, and (2) the probe’s accuracy at predicting scene embeddings. Fig. 9 shows a strong positive correlation: models that generalize well are precisely those whose embeddings are well-approximated by multiplicative structure. This suggests that generalizing models are internally implementing something akin to multiplicative binding—a simple functional form that composes concept representations systematically. The correlation weakens slightly for the largest object space ($|O| = 125k$), but the pattern remains clear: the multiplicative form is sufficient to capture how concepts combine, which is why it generalizes to unseen combinations.

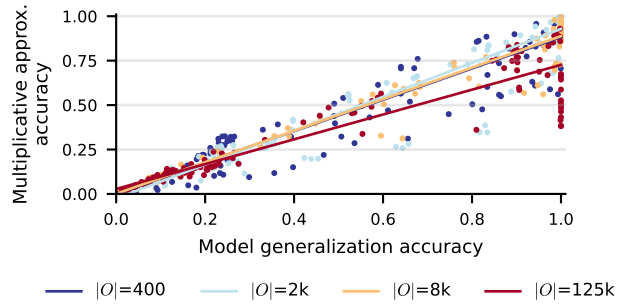


Figure 9. Generalization correlates with multiplicative structure. Each point is a trained model; x-axis shows object recognition accuracy on scenes composed of held-out objects, y-axis shows how well a multiplicative probe (trained on 50% of objects) approximates scene embeddings. Colors indicate object space size. Models that generalize also admit simpler (multiplicative) binding.

Takeaway §5.4: Generalizable binding corresponds to multiplicative structure in the embedding space: models whose embeddings admit simple multiplicative composition are precisely those that generalize to unseen object combinations.

6. Discussion and conclusion

In this work we studied concept binding by analyzing the geometry of multi-object embeddings and distinguishing concept recognition from object recognition. In CLIP, scene embeddings show a clear additive object-level structure (supporting uni-modal decodability and direct embedding edits), but the concept-to-object mapping is high-complexity and combination-specific, which blocks generalization and makes cross-modal alignment brittle. In controlled transformer dual-encoders trained from scratch, binding generalization emerges with enough coverage, and the generalizing mapping is best explained by low-complexity structure with multiplicative interactions between concepts.

7. Impact statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Abbasi, R., Rohban, M. H., and Baghshah, M. S. Deciphering the role of representation disentanglement: Investigating compositional generalization in clip models. *arXiv preprint arXiv:2407.05897*, 2024. 2
- Assouel, R., Campbell, D., Bengio, Y., and Webb, T. Visual symbolic mechanisms: Emergent symbol processing in vision language models. *arXiv preprint arXiv:2506.15871*, 2025. 2
- Berasi, D., Farina, M., Mancini, M., Ricci, E., and Strisciuglio, N. Not only text: Exploring compositionality of visual representations in vision-language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 24917–24927, 2025. 2
- Bergsma, S., Dey, N., Gosal, G., Gray, G., Soboleva, D., and Hestness, J. Straight to zero: Why linearly decaying the learning rate to zero works best for llms, 2025. URL <https://arxiv.org/abs/2502.15938>. 15
- Blier, L. and Ollivier, Y. The description length of deep learning models, 2018. URL <https://arxiv.org/abs/1802.07044>. 5
- Borg, I. and Groenen, P. J. *Modern multidimensional scaling: Theory and applications*. Springer, 2005. 3
- Campbell, D., Rane, S., Giallanza, T., De Sabbata, N., Ghods, K., Joshi, A., Ku, A., Frankland, S. M., Griffiths, T. L., Cohen, J. D., et al. Understanding the limits of vision language models through the lens of the binding problem. *arXiv preprint arXiv:2411.00238*, 2024. 2, 19
- Dittadi, A., Träuble, F., Locatello, F., Wüthrich, M., Agrawal, V., Winther, O., Bauer, S., and Schölkopf, B. On the transfer of disentangled representations in realistic settings, 2021. URL <https://arxiv.org/abs/2010.14407>. 2
- Elmoznino, E., Jiralerspong, T., Bengio, Y., and Lajoie, G. Towards a formal theory of compositionality, 2025. URL <https://openreview.net/forum?id=hKMPz3wkPV>. 5
- Feng, J. and Steinhardt, J. How do language models bind entities in context?, May 2024. URL <https://doi.org/10.48550/arXiv.2310.17191>. 2
- Feng, J., Russell, S., and Steinhardt, J. Monitoring latent world states in language models with propositional probes, June 2024. URL <http://arxiv.org/abs/2406.19501>. 2
- Greff, K., Van Steenkiste, S., and Schmidhuber, J. On the binding problem in artificial neural networks. *arXiv preprint arXiv:2012.05208*, 2020. 2
- Gurung, B., Hoffmann, D. T., and Brox, T. Common data properties limit object-attribute binding in clip. In *DAGM German Conference on Pattern Recognition*, pp. 353–368. Springer, 2025. 2
- Hsieh, C.-Y., Zhang, J., Ma, Z., Kembhavi, A., and Krishna, R. Sugarcreeper: Fixing hackable benchmarks for vision-language compositionality. *Advances in neural information processing systems*, 36, 2024. 2
- Huang, L., Li, Y., Chang, Y., Salehi, S., and Kording, K. The binding problem in vision models: Geometric, functional, and behavioral approaches. In *NeurIPS 2025 Workshop on Symmetry and Geometry in Neural Representations*, 2025. URL <https://openreview.net/forum?id=rUK0P1Ejxl>. 2
- Jarvis, D., Klein, R., Rosman, B., and Saxe, A. M. On the specialization of neural modules, 2024. URL <https://arxiv.org/abs/2409.14981>. 2
- Jiang, Y., Rajendran, G., Ravikumar, P., Aragam, B., and Veitch, V. On the origins of linear representations in large language models. *arXiv preprint arXiv:2403.03867*, 2024. 2
- Kamath, A., Hessel, J., and Chang, K.-W. Text encoders bottleneck compositionality in contrastive vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4933–4944, 2023. 2
- Kang, R., Song, Y., Gkioxari, G., and Perona, P. Is clip ideal? no. can we fix it? yes! *ICCV*, 2025. 2
- Kempf, E., Schrod, S., Argus, M., and Brox, T. When and how does clip enable domain and compositional generalization?, 2025. URL <https://arxiv.org/abs/2502.09507>. 2
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>. 16
- Koishigarina, D., Uselis, A., and Oh, S. J. Clip behaves like a bag-of-words model cross-modally but not unimodally, February 2025. URL <https://doi.org/10.48550/arXiv.2502.03566>. 1, 2, 3

- Lewis, M., Nayak, N., Yu, P., Merullo, J., Yu, Q., Bach, S., and Pavlick, E. Does clip bind concepts? probing compositionality in large image models. In *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 1487–1500, 2024. 1, 2, 6
- Li, S., Koh, P. W., and Du, S. S. Exploring how generative mllms perceive more than clip with the same vision encoder. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10101–10119, 2025. 2
- Li, Y., Salehi, S., Ungar, L., and Kording, K. P. Does object binding naturally emerge in large pretrained vision transformers?, 2026. URL <https://arxiv.org/abs/2510.24709>. 2
- Liang, Q., Qian, D., Ziyin, L., and Fiete, I. Compositional generalization via forced rendering of disentangled latents, 2025. URL <https://arxiv.org/abs/2501.18797>. 2
- Lippl, S. and Stachenfeld, K. When does compositional structure yield compositional generalization? a kernel theory, 2025. URL <https://arxiv.org/abs/2405.16391>. 2
- Ma, Z., Hong, J., Gul, M. O., Gandhi, M., Gao, I., and Krishna, R. Crepe: Can vision-language foundation models reason compositionally? In *CVPR*, pp. 10910–10921, 2023. 2
- Mahajan, D., Pezeshki, M., Arnal, C., Mitliagkas, I., Ahuja, K., and Vincent, P. Compositional risk minimization, 2025. URL <https://arxiv.org/abs/2410.06303>. 2
- Montero, M. L., Ludwig, C. J., Costa, R. P., Malhotra, G., and Bowers, J. The role of disentanglement in generalisation. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=qbH974jKUVy>. 2
- Okawa, M., Lubana, E. S., Dick, R., and Tanaka, H. Compositional abilities emerge multiplicatively: Exploring diffusion models on a synthetic task. *Advances in Neural Information Processing Systems*, 36:50173–50195, 2023. 2
- Park, K., Choe, Y. J., and Veitch, V. The linear representation hypothesis and the geometry of large language models. *arXiv preprint arXiv:2311.03658*, 2023. 2
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pp. 8748–8763. PMLR, 2021. URL <https://proceedings.mlr.press/v139/radford21a.html>. 1, 6, 13
- Schott, L., von Kügelgen, J., Träuble, F., et al. Visual representation learning does not generalize strongly within the same domain, 2022. URL <https://arxiv.org/abs/2107.08221>. 2
- Tang, Y., Yamada, Y., Zhang, Y., and Yildirim, I. When are lemons purple? the concept association bias of vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 14333–14348, 2023. 1, 2
- Tong, S., Jones, E., and Steinhardt, J. Mass-producing failures of multimodal systems with language models. *NIPS*, 36, 2024a. 2
- Tong, S., Liu, Z., Zhai, Y., Ma, Y., LeCun, Y., and Xie, S. Eyes wide shut? exploring the visual shortcomings of multimodal llms. In *CVPR*, pp. 9568–9578, 2024b. 2
- Trager, M., Perera, P., Zancato, L., Achille, A., Bhatia, P., and Soatto, S. Linear spaces of meanings: compositional structures in vision-language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 15395–15404, 2023. 2
- Treisman, A. Feature binding, attention and object perception. *Philosophical Transactions of the Royal Society of London. Series B: Biological Sciences*, 353(1373):1295–1306, 1998. 1
- Treisman, A. M. and Gelade, G. A feature-integration theory of attention. *Cognitive Psychology*, 12(1):97–136, 1980. 1
- Uselis, A., Dittadi, A., and Oh, S. J. Does data scaling lead to visual compositional generalization? In *Forty-second International Conference on Machine Learning*, 2025a. URL <https://openreview.net/forum?id=M2WMMUwuh5>. 2
- Uselis, A., Dittadi, A., and Oh, S. J. Does data scaling lead to visual compositional generalization?, 2025b. URL <https://arxiv.org/>. 2
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>. 6
- Wang, H., Si, H., Shao, H., and Zhao, H. Enhancing compositional generalization via compositional feature alignment. *arXiv preprint arXiv:2402.02851*, 2024. 2

Yuksekgonul, M., Bianchi, F., Kalluri, P., Jurafsky, D., and
Zou, J. When and why vision-language models behave
like bags-of-words, and what to do about it? In *ICLR*,
2023. [1](#), [2](#)

Appendix

A	Notation summary	13
B	Details on the controlled models	13
B.1	Scene space and tokenization	13
B.2	Encoders	14
B.3	Queries, labels, and loss	14
B.4	Optimization and controlled scenarios	15
C	Experimental detail	15
C.1	CLIP geometry experiments	15
C.2	Text encoder decomposition	16
C.3	Details on approximating pre-trained models' binding function	16
D	Interventions	17
E	Editing scene embeddings with object embeddings	18

A. Notation summary

Table 5. Core notation used in the main text.

Symbol	Meaning
C	Number of concepts per object; $i \in \{1, \dots, C\}$ indexes concepts.
$\mathcal{C}_i$	Set of values for concept i ; V denotes $ \mathcal{C}_i $ when uniform.
$\mathcal{C}$	Concept space $\mathcal{C}_1 \times \dots \times \mathcal{C}_C$.
$\mathbf{o} = (c_1, \dots, c_C)$	Object (concept-value tuple); $c_i \in \mathcal{C}_i$.
$O_{\max}$	Maximum number of objects in a scene.
m	Number of objects in a scene.
$\mathcal{S}_m$	Set of ordered m -object scenes; $\mathcal{S} = \bigcup_{m=1}^{O_{\max}} \mathcal{S}_m$.
$\mathbf{s}$	Scene (tuple of objects).
$\mathbf{x}_{\mathbf{s}}$	Datapoint (e.g., image) depicting scene $\mathbf{s}$.
$\mathcal{X}, \mathcal{Y}$	Input space and query space.
f, g	Input encoder $f: \mathcal{X} \rightarrow \mathbb{R}^d$ and query encoder $g: \mathcal{Y} \rightarrow \mathbb{R}^d$.
d	Embedding dimension.
$s(\mathbf{x}_{\mathbf{s}}, \mathbf{y})$	Cosine similarity between scene and query embeddings.
$\mathbf{y}_{i,v}, \mathbf{y}_{\mathbf{o}}$	Query for concept i value v ; query for object $\mathbf{o}$.
$V_i(\mathbf{s})$	Set of concept- i values present in scene $\mathbf{s}$.
$O(\mathbf{s})$	Set of objects present in scene $\mathbf{s}$.
$\mathbf{u}_{\mathbf{o}}$	Object embedding; $\mathbf{u}_{c_j,i}$ is the embedding of the value of concept i for object $\mathbf{o}_j$.
$\tilde{\mathbf{z}}$	Counterfactual (edited) scene embedding.
$R^2, \mathbf{f}$	Variance explained; $\mathbf{f}$ is the mean scene embedding.
$\pi_{\mathbf{c}}, \pi_{\mathbf{o}}$	Permutations of concept/object IDs in control ablations.
$\mathbf{W}, l$	Linear model weights and input dimension in additive regressions.
$g(\mathbf{o}_1, \mathbf{o}_2)$	Binding-function approximator (MLP) mapping objects to embeddings.

B. Details on the controlled models

We train controlled models from scratch on synthetic multi-object scenes represented as token sequences. We use two transformer encoders: one for scenes and one for queries (either a single concept value or an entire object). We optimize a retrieval objective that makes matching scene-query pairs more similar than mismatched pairs (akin to CLIP’s contrastive loss (Radford et al., 2021)). Because both scenes and queries are generated procedurally, we can vary the object-space size V^C , the maximum number of objects per scene $O_{\max}$, and whether evaluation uses held-out concept combinations. Table 6 summarizes the key parameters used throughout.

Table 6. Notation and key configuration parameters for the controlled models.

Symbol	Code	Meaning
C	num_concepts	Number of concepts per object.
V	num_vals_per_concept	Number of values per concept.
$O_{\max}$	max_num_objects	Maximum number of objects per scene.
$L_{\max}$	max_scene_size	Maximum sequence length, $L_{\max} = (C + 2)O_{\max} + 1$.
d_{model}	model_d_model	Transformer width.
H	model_num_heads	Number of attention heads.
L	model_num_layers	Number of transformer layers.
d_{out}	model_d_out	Output embedding dimension.
K_{concept}	num_concept_values_take_max	Max number of concept queries per step.
K_{obj}	num_object_values_take_max	Max number of object queries per step.
M_{perturb}	num_negative_objs_per_pos	Perturbation negatives per positive object.
M_{swap}	num_swap_obj_negatives_per_scene	Swap negatives generated per scene.
ρ_{train}	train_object_fraction	Fraction of objects available during training (object-space split).
s_{split}	object_split_seed	Seed for the object-space split.
sim	sim_type	Similarity function (cos or dot).

B.1. Scene space and tokenization

We parameterize the synthetic environment by the number of concepts C , the number of values per concept V , and the maximum number of objects per scene $O_{\max}$ (max_num_objects). Each object is represented as a C -tuple of discrete

concept values $\mathbf{v} = (v_1, \dots, v_C)$ with $v_c \in \{0, \dots, V - 1\}$, and a scene is an ordered list of $n \in \{1, \dots, O_{\max}\}$ such objects.

Scene sampling. We draw the scene length $n \sim \text{Unif}\{1, \dots, O_{\max}\}$ and then sample n objects independently. By default, each component v_c is drawn uniformly from $\{0, \dots, V - 1\}$. To implement train/test splits, we instead sample objects from a restricted list `allowed_objects`.

Tokenization. Table 7 summarizes the sequence format. We map each concept/value pair (c, v) to a token id $t(c, v) = (c - 1)V + v$, yielding CV concept-value tokens, and add four extra tokens: SOO (start-of-object), EOO (end-of-object), EOS (end-of-scene), and PAD. An object $\mathbf{v} = (v_1, \dots, v_C)$ is encoded as

$$[\text{SOO}, t(1, v_1), \dots, t(C, v_C), \text{EOO}],$$

and a scene is the concatenation of its objects followed by EOS, padded to the fixed length

$$L_{\max} = (C + 2)O_{\max} + 1.$$

Scenes are generated on the fly by an “infinite” iterator, so training is naturally described in gradient steps rather than epochs.

Table 7. Token IDs and sequence formats used for scenes and probe queries.

Component	Definition
Concept-value token	$t(c, v) = (c - 1)V + v$ for concept $c \in \{1, \dots, C\}$, value $v \in \{0, \dots, V - 1\}$.
Special tokens	SOO = CV , EOO = $CV + 1$, EOS = $CV + 2$, PAD = $CV + 3$.
Object sequence	$[\text{SOO}, t(1, v_1), \dots, t(C, v_C), \text{EOO}]$.
Scene sequence	Concatenate n objects, append EOS, pad with PAD to length $L_{\max}$.
Concept query	$[t(c, v), \text{EOS}]$ padded to $L_{\max}$.
Object query	$[\text{SOO}, t(1, v_1), \dots, t(C, v_C), \text{EOO}, \text{EOS}]$ padded to $L_{\max}$.

Example. With $C = 2$ and $V = 10$, the object $\mathbf{v} = (3, 7)$ (i.e., $v_1 = 3, v_2 = 7$) is encoded as $[\text{SOO}, 3, 17, \text{EOO}]$ since $t(1, 3) = 3$ and $t(2, 7) = 10 + 7 = 17$.

B.2. Encoders

Scenes and queries are embedded by two transformer encoders with the same architecture (`SceneEncoder`) but independent parameters: a *scene encoder* f_{scene} and a *probe encoder* f_{probe} . Each maps a padded length- $L_{\max}$ token sequence to a single embedding via token embeddings, positional encoding, and an L -layer Transformer encoder with H attention heads (masking PAD). The scene encoder outputs $\mathbf{s} \in \mathbb{R}^{d_{\text{out}}}$. The probe encoder outputs $\mathbf{q} \in \mathbb{R}^{d_{\text{out}}}$.

For a minibatch of B scenes and T queries, let $\mathbf{S} \in \mathbb{R}^{B \times d_{\text{out}}}$ and $\mathbf{Q} \in \mathbb{R}^{T \times d_{\text{out}}}$ collect their embeddings row-wise. We form a logit matrix $\mathbf{Z} \in \mathbb{R}^{B \times T}$. We then optimize the embeddings $\hat{\mathbf{x}} = \mathbf{x} / (\|\mathbf{x}\|_2 + 10^{-6})$ and define $\hat{\mathbf{S}}, \hat{\mathbf{Q}}$ by row-wise normalization. We use $\tau = \exp(\tilde{\tau})$ as a learned temperature parameter, akin to CLIP (clamped to $\tau \leq 100$).

B.3. Queries, labels, and loss

At each training step we sample a batch of scenes and construct a set of query sequences. We form a multi-hot label matrix $\mathbf{Y} \in \{0, 1\}^{B \times T}$, where $Y_{ij} = 1$ if query j matches scene i . We use two query types:

1. **Concept retrieval.** Concept queries specify a single concept value. We collect the unique concept-value tokens that appear in the batch (excluding SOO/EOO/EOS/PAD), subsample up to `num_concept_values_take_max`, and tokenize each as a concept query.
2. **Object retrieval.** Object queries specify a full object. We parse object blocks from the tokenized scenes, take the set of unique objects present in the batch, subsample up to 1024, and tokenize each as an object query. We then augment the query set with hard negatives (random/perturbed objects and attribute swaps).

Concept queries probe whether the model can retrieve scenes based on single attributes, while object queries probe multi-attribute binding.

Given logits $\mathbf{Z}$ and labels $\mathbf{Y}$, we treat multiple positives by normalizing each row to a target distribution $\mathbf{P}_i = \mathbf{Y}_i / (\sum_j \mathbf{Y}_{ij} + 10^{-10})$ and minimizing the cross-entropy

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \sum_{j=1}^T \mathbf{P}_{ij} \log \text{softmax}(\mathbf{Z}_{i\cdot})_j.$$

Scenes with $\sum_j \mathbf{Y}_{ij} = 0$ (none of the sampled queries match) have $\mathbf{P}_i = 0$ and contribute zero loss. When both concept and object queries are enabled we sum their losses and backpropagate through both encoders and $\tilde{\tau}$.

B.4. Optimization and controlled scenarios

Training uses AdamW over the parameters of both encoders and $\tilde{\tau}$, with a linear warmup (first 10% of steps) followed by linear decay (Bergsma et al., 2025). The code can optionally take multiple optimizer steps per sampled minibatch (num_grad_steps). Table 8 lists the default hyperparameters and numerical-stability choices.

Table 8. Training hyperparameters and numerical-stability choices (defaults unless stated otherwise).

Component	Setting
Training steps	$T = 50,000$ gradient steps
Batch size	$B = 1024$ scenes per step
Optimizer	AdamW over both encoders and $\tilde{\tau}$
Learning rate	3×10^{-4}
Weight decay	0.001
LR schedule	Linear warmup for $0.1T$ steps, then linear decay to 0
Temperature init/clamp	Initialize $\tilde{\tau} = 1.15$, use $\tau = \exp(\tilde{\tau})$ clamped to $\tau \leq 100$

C. Experimental detail

C.1. CLIP geometry experiments

This section specifies how we compute the *retrieval* and *probing* metrics reported for the Level-I/II decomposition experiments (Tab. 1) and for the intervention experiments (§4.1). To evaluate whether binding-relevant information is present in embeddings *within a modality*, we use *trained linear probes* on frozen embeddings.

Probing metric (concept/object recognition accuracy). Let $\mathbf{z}_s = f(\mathbf{x}_s) \in \mathbb{R}^d$ denote an embedding of a scene s (image or text, depending on the encoder). For each concept $i \in [C]$, we train a linear probe $h_i(\mathbf{z}) = \mathbf{W}_i \mathbf{z} + \mathbf{b}_i \in \mathbb{R}^{|C_i|}$ that outputs logits over the values of concept i . We also train an object probe $h_{\text{obj}}(\mathbf{z}) = \mathbf{W}_{\text{obj}} \mathbf{z} + \mathbf{b}_{\text{obj}} \in \mathbb{R}^{|C|}$ that outputs logits over the object space (concept tuples). Because scenes contain multiple objects, the target for a probe is generally *multi-hot* (multiple values/objects are present). We train probes on frozen embeddings by minimizing cross-entropy to the normalized multi-positive target distribution (as in §B).

At evaluation time, we evaluate using the metrics in Def. 3.3 and Def. 3.4. For concept i , let $h_i(\mathbf{z}_s)_v$ be the logit for value $v \in \mathcal{C}_i$. We count the scene as correct for concept i if

$$\min_{v \in V_i(s)} h_i(\mathbf{z}_s)_v > \max_{v \in \mathcal{C}_i \setminus V_i(s)} h_i(\mathbf{z}_s)_v.$$

Concept recognition accuracy is the mean of this indicator over scenes and then averaged across concepts. Object recognition accuracy is computed analogously from $h_{\text{obj}}(\mathbf{z}_s)$, using the set of objects $O(s)$.

Retrieval metric (nearest-neighbor scene retrieval). For a fixed dataset of two-object scenes $\{\mathbf{s}_n\}_{n=1}^N$, we precompute their embeddings $\mathbf{z}_n = f(\mathbf{x}_{\mathbf{s}_n})$. Given a predicted embedding $\tilde{\mathbf{z}}_n$ (e.g., a reconstruction from object means, or an edited embedding from §4.1), retrieval is treated as a nearest-neighbor problem in embedding space:

$$\hat{n} = \arg \max_{m \in [N]} \frac{\langle \tilde{\mathbf{z}}_n, \mathbf{z}_m \rangle}{\|\tilde{\mathbf{z}}_n\|_2 \|\mathbf{z}_m\|_2}.$$

Retrieval is correct if the nearest neighbor corresponds to the intended scene. When multiple scenes share the same underlying object tuple (e.g., because of positional variation), we treat any scene with the intended object tuple as correct. We report top-1 accuracy over all evaluated scenes.

Reconstruction metric. For the R^2 score, we measure how much variance is explained by the addition of the two object embeddings. In other words, the score measures how well V^2 vectors can reconstruct all of the V^4 embeddings. The R^2 score is defined as

$$R^2 := 1 - \frac{\sum_{\mathbf{x}_s} \|f(\mathbf{x}_s) - \mathbf{u}_{o_0} - \mathbf{u}_{o_1}\|_2^2}{\sum_{\mathbf{x}_s} \|f(\mathbf{x}_s) - \mathbf{f}\|_2^2}. \quad (7)$$

C.2. Text encoder decomposition

Setup. We generate a synthetic caption dataset for scenes by specifying a finite *concept space*

$$\mathcal{C} := \mathcal{C}_1 \times \mathcal{C}_2,$$

where $\mathcal{C}_1$ contains *adjective* tokens (e.g., color/size/material) and $\mathcal{C}_2$ contains *noun* tokens (e.g., shape/category). In our experiments we generate 10,000 two-object scenes plus 100 single-object scenes; each scene is a list of objects, and each object is represented by a pair of integer indices (i_1, i_2) selecting tokens from $\mathcal{C}_1$ and $\mathcal{C}_2$.

Caption construction. For each scene, we linearize objects into a string of the form “a ⟨attr⟩ ⟨obj⟩ and a ⟨attr⟩ ⟨obj⟩”. The following snippet shows the exact procedure used.

```
concept_space = [
    ["red", "blue", "green", "yellow", "purple", "orange", "pink", "brown", "gray", "black",
     "small", "large", "tiny", "round", "sharp", "rough", "bumpy", "smooth", "matte", "glossy"],
    ["circle", "square", "triangle", "star", "heart", "spiral", "sphere", "cylinder", "cone", "pyramid",
     "car", "cat", "dog", "man", "woman", "tree", "flower", "rock", "bird", "fish"],
]

captions_dataset = []

for scene in scene_dataset:
    scene_caption = "a"
    for obj_i, obj in enumerate(scene):
        for concept_i, concept in enumerate(obj):
            scene_caption += f"_{concept_space[concept_i][concept]}"
        if obj_i != len(scene) - 1:
            scene_caption += "_and_a"
    captions_dataset.append(scene_caption)
```

C.3. Details on approximating pre-trained models’ binding function

This section provides experimental details for the binding function approximation experiments on pre-trained models (§5.1). Our aim is to determine whether CLIP’s binding function *can* be approximated by a learnable function, rather than to find the most efficient approximator. To ensure that any failure to generalize reflects genuine complexity of the binding function rather than insufficient model capacity, we deliberately over-parameterize the approximator and sweep over a wide range of configurations.

MLP architectures. We use ReLU MLPs that take as input a concatenation of one-hot encodings of concept indices for both objects in the scene (i.e., $2 \times C \times V$ binary features, where C is the number of concepts and V is the number of values per concept). We experiment with 1-layer MLPs with hidden widths $\in \{64, 128, 256, 512, 1024, 2048, 4096\}$;

The output dimension matches the embedding dimension d of the encoder (e.g., $d = 512$ for CLIP ViT-B/32). Results reported in the main text correspond to the *best* performance achieved across all architectures for each training fraction, ensuring that we report an upper bound on what the approximator family can achieve.

Training. We use the Adam (Kingma & Ba, 2017) optimizer with learning rates in $\{10^{-2}, 10^{-3}, 10^{-4}\}$, batch size 4096, for 10,000 steps. We select the best configuration based on test object accuracy. We split the V^C possible objects into train/test sets, varying training coverage over $\{10\%, 20\%, \dots, 90\%\}$. Test scenes contain only held-out objects, i.e., objects whose concept configurations never appeared during training.

Evaluation. We apply linear probes (trained on true embeddings) to the predicted embeddings and measure concept/object recognition accuracy. This tests whether predicted embeddings preserve functional binding properties, not just reconstruction fidelity. For text: we use synthetic captions (§C.2); for images: CLEVR, CLEVR-2D, PUG:SPARE. We report the best accuracy across all MLP architectures and learning rates for each configuration.

Table 9. Decodability results. We report probe accuracy after subtracting components from the embedding (remove concepts/objects) or their permuted-ID controls, for the image encoder and its random-init counterpart for nonpositional / positional variations.

Intervention	Conc. ($\uparrow$)	Obj. ($\uparrow$)
CLIP ViT-B/32 (CLEVR-2D)		
None	1.00 / 1.00	0.99 / 0.99
Subtract concepts	0.32 / 0.30	0.78 / 0.79
Subtract objects	0.28 / 0.29	0.08 / 0.10
Permute concepts	0.83 / 0.83	0.95 / 0.95
Permute objects	0.92 / 0.91	0.86 / 0.86
CLIP ViT-B/32 (CLEVR)		
None	1.00 / 1.00	0.93 / 0.93
Subtract concepts	0.28 / 0.27	0.77 / 0.79
Subtract objects	0.26 / 0.23	0.01 / 0.03
Permute concepts	0.83 / 0.81	0.91 / 0.93
Permute objects	0.93 / 0.91	0.84 / 0.82
DINOv2 ViT-B/14 (CLEVR-2D)		
None	1.00 / 1.00	0.97 / 0.96
Subtract concepts	0.26 / 0.27	0.79 / 0.79
Subtract objects	0.25 / 0.23	0.03 / 0.06
Permute concepts	0.79 / 0.78	0.94 / 0.93
Permute objects	0.91 / 0.90	0.85 / 0.86
DINOv2 ViT-B/14 (CLEVR)		
None	0.99 / 0.99	0.92 / 0.92
Subtract concepts	0.20 / 0.20	0.61 / 0.65
Subtract objects	0.18 / 0.20	0.01 / 0.00
Permute concepts	0.77 / 0.76	0.92 / 0.92
Permute objects	0.87 / 0.88	0.83 / 0.81

D. Interventions

In §4.1 we test whether the Level-I decomposition is actionable by performing object-replacement edits in embedding space. This appendix specifies (i) how we estimate object components $\mathbf{u}_o$ and (ii) how we score the edited embeddings.

Estimating object components. For a fixed encoder and dataset of two-object scenes, we estimate an object component $\mathbf{u}_o$ in three ways:

1. *Scene averaging (avg)*: average embeddings of all two-object scenes in which o appears (after mean-centering over the two-object scene set).
2. *Scene averaging + position (avg+pos)*: same as above, but maintain separate averages for each slot in the scene (e.g., left/right object), yielding position-specific components.
3. *Single-object scenes (single-obj)*: use embeddings of single-object scenes that contain only o , when such scenes are available.

Metrics. We use the same retrieval and probing metrics defined in §C.1. For interventions, the *target* of an edited embedding is the intended counterfactual scene (or, when multiple scenes share the same object tuple due to positional variation, any scene with that object tuple is treated as correct).

Permuted-ID controls. To ensure that effects are not explained by subtracting an arbitrary but similarly-sized signal, we also run *permuted-ID* controls for both concept and object ablations. Concretely, let $N_c := \sum_{i=1}^C |\mathcal{C}_i|$ be the total number of concept values (often $N_c = C \cdot V$), and let $N_o := |\mathcal{C}| = \prod_{i=1}^C |\mathcal{C}_i|$ be the number of objects (concept tuples). We sample uniform random permutations $\pi_c : [N_c] \rightarrow [N_c]$ and $\pi_o : [N_o] \rightarrow [N_o]$. When applying π_c , we treat each concept value (i, v) as a unique ID in $[N_c]$; when applying π_o , we treat each object $o \in \mathcal{C}$ as a unique ID in $[N_o]$. We then form the same

Table 10. **Decodability results.** We report probe accuracy after subtracting components from the embedding (remove concepts/objects) or their permuted-ID controls, for both text and image encoders and their random-init counterparts for nonpositional / positional variations.

Model	Intervention	Text		Image	
		Conc. ($\uparrow$)	Obj. ($\uparrow$)	Conc. ($\uparrow$)	Obj. ($\uparrow$)
Random	None	0.95 / 0.95	0.59 / 0.60	0.78 / 0.78	0.71 / 0.71
	Subtract concepts	0.02 / 0.02	0.27 / 0.43	0.06 / 0.08	0.66 / 0.68
	Subtract objects	0.01 / 0.01	0.00 / 0.00	0.04 / 0.05	0.00 / 0.01
	Permute concepts	0.88 / 0.88	0.69 / 0.75	0.80 / 0.81	0.78 / 0.76
	Permute objects	0.83 / 0.84	0.93 / 0.98	0.78 / 0.80	0.88 / 0.88
CLIP-B/32	None	1.00 / 1.00	1.00 / 1.00	0.95 / 0.95	0.88 / 0.88
	Subtract concepts	0.07 / 0.07	0.99 / 0.99	0.06 / 0.08	0.73 / 0.73
	Subtract objects	0.05 / 0.05	0.02 / 0.03	0.04 / 0.04	0.00 / 0.00
	Permute concepts	0.92 / 0.93	1.00 / 1.00	0.84 / 0.84	0.91 / 0.91
	Permute objects	0.96 / 0.96	1.00 / 1.00	0.82 / 0.83	0.92 / 0.93
DINOv2-B/14	None	-	-	0.95 / 0.94	0.79 / 0.79
	Subtract concepts	-	-	0.04 / 0.04	0.62 / 0.70
	Subtract objects	-	-	0.03 / 0.04	0.00 / 0.00
	Permute concepts	-	-	0.77 / 0.77	0.80 / 0.79
	Permute objects	-	-	0.83 / 0.83	0.87 / 0.87

residual embeddings as in the main text, but with permuted components, e.g.,

$$f(\mathbf{x}_s) - \sum_i \mathbf{u}_{\pi_c(c_{1,i})} - \sum_i \mathbf{u}_{\pi_c(c_{2,i})} \quad \text{or} \quad f(\mathbf{x}_s) - \mathbf{u}_{\pi_o(o_1)} - \mathbf{u}_{\pi_o(o_2)}.$$

We evaluate these permuted residuals with the same probes.

E. Editing scene embeddings with object embeddings

We provide additional results for the object-replacement interventions introduced in §4.1. These experiments test whether object embeddings can be reused across scenes to produce counterfactual scene embeddings via simple arithmetic.

Setup. Given a two-object scene $s = (o_1, o_2)$ with embedding $f(\mathbf{x}_s)$, we construct an edited embedding by replacing o_1 with a counterfactual object o'_1 :

$$\tilde{z} = f(\mathbf{x}_s) - k \mathbf{u}_{o_1} + k \mathbf{u}_{o'_1}, \quad (8)$$

where k controls the strength of the intervention. Setting $k = 0$ leaves the embedding unchanged, while larger k increases the contribution of the replacement. In practice, performance varies with k , and we report full curves over k in Figure 10. Unless stated otherwise, results in Tab. 11 use $k = 1.0$.

Object embeddings $\mathbf{u}_o$ are estimated in three ways: (i) **obj-avg**, by averaging embeddings of all scenes containing the object; (ii) **pos-avg**, by averaging with position-specific grouping; and (iii) **single**, by using embeddings of single-object scenes when available.

We distinguish between two settings. In the *different-concepts* setting, the two objects in the scene do not share any concept values (e.g., a red cube and a blue sphere). In the *shared-concepts* setting, the objects share at least one concept value (e.g., a red cube and a red sphere). The shared-concepts setting is more challenging because the two objects occupy overlapping concept directions in the representation.

Results. Table 11 reports probing and retrieval performance for object replacement interventions at full strength ($k = 1.0$) across datasets and models. In most settings, replacing an object embedding yields scene embeddings that support both object probing and retrieval consistent with the intended counterfactual scene. Object embeddings obtained via scene averaging and position-specific averaging perform similarly across datasets, indicating that positional effects are limited. When available, embeddings derived from single-object scenes also enable effective edits, though with reduced performance in some settings.

Performance drops substantially when the two objects in the scene share concepts, particularly for retrieval. This effect is consistent across datasets and encoders. A plausible explanation is feature interference: when objects share concept values,

How can embedding models bind concepts?

Dataset	Model	Concepts	Probing ($\uparrow$)			Retrieval ($\uparrow$)		
			obj-avg	pos-avg	single	obj-avg	pos-avg	single
Text	CLIP ViT-B/32	Different	0.99	0.99	0.69	0.99	0.99	0.96
Text	CLIP ViT-B/32	Shared	0.86	0.86	0.23	0.73	0.73	0.10
CLEVR	CLIP ViT-B/32	Different	0.98	0.98	0.86	1.00	1.00	0.97
CLEVR	CLIP ViT-B/32	Shared	0.77	0.78	0.25	0.38	0.39	0.08
CLEVR	CLIP ViT-L/14	Different	0.98	0.98	0.90	0.98	0.99	0.98
CLEVR	CLIP ViT-L/14	Shared	0.80	0.80	0.28	0.10	0.11	0.05
CLEVR	DINO ViT-B/14	Different	0.97	0.98	0.77	0.89	0.95	0.86
CLEVR	DINO ViT-B/14	Shared	0.95	0.94	0.29	0.30	0.42	0.13
CLEVR-2D	CLIP ViT-B/32	Different	0.98	0.98	0.92	0.99	0.99	0.97
CLEVR-2D	CLIP ViT-B/32	Shared	0.68	0.67	0.26	0.14	0.14	0.05
CLEVR-2D	CLIP ViT-L/14	Different	0.98	0.98	0.90	0.95	0.95	0.90
CLEVR-2D	CLIP ViT-L/14	Shared	0.72	0.72	0.36	0.13	0.14	0.07
CLEVR-2D	DINO ViT-B/14	Different	0.97	0.98	0.72	0.97	0.97	0.88
CLEVR-2D	DINO ViT-B/14	Shared	0.87	0.88	0.29	0.14	0.16	0.03
PUG:SPARE	CLIP ViT-B/32	Different	0.94	0.95	–	0.86	0.94	–
PUG:SPARE	CLIP ViT-L/14	Different	0.96	0.96	–	0.58	0.74	–
PUG:SPARE	DINO ViT-B/14	Different	0.97	0.97	–	0.48	0.76	–

Table 11. Intervention results summary at $k = 1.0$. Object embeddings are constructed by object averaging (obj-avg), position-wise averaging (pos-avg), or from single-object scenes (single).

their contributions overlap in representation space, making it harder to selectively remove and insert a single object without affecting the other (Campbell et al., 2024).

We further analyze the role of intervention strength in Figs. 10 to 12. Varying the intervention coefficient k reveals that performance typically improves with stronger interventions before saturating. However, in shared-concept settings, optimal performance is often achieved at smaller values of k , suggesting that partial edits mitigate interference effects.

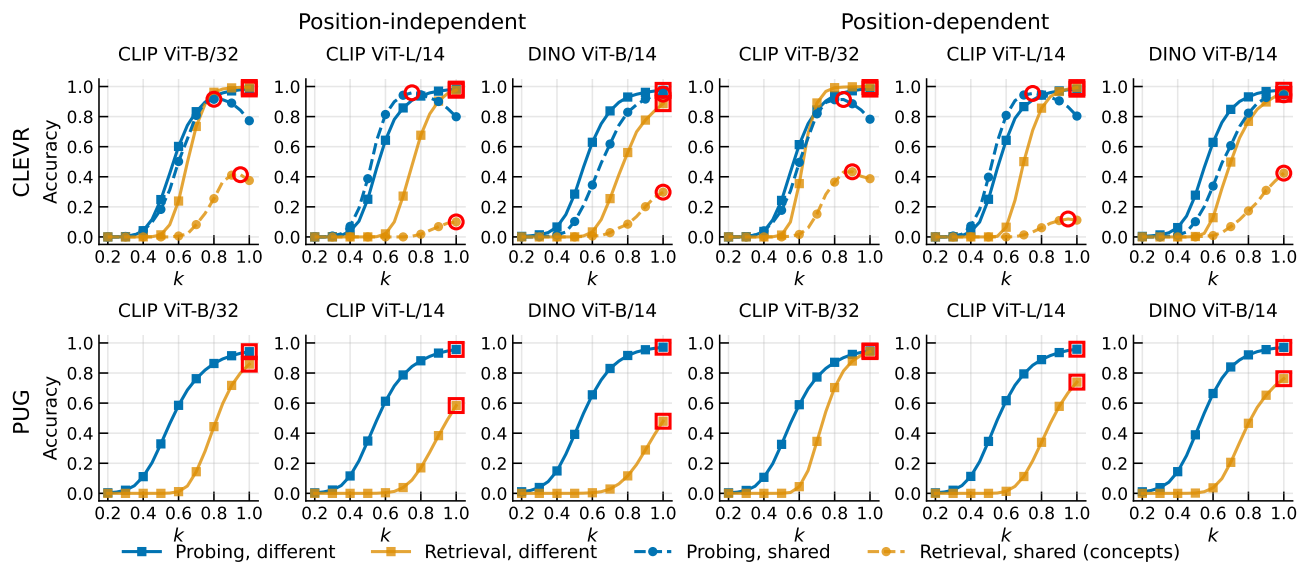


Figure 10. Effect of intervention strength on object replacement for CLEVR and PUG:SPARE with position-independent and position-dependent averaging.

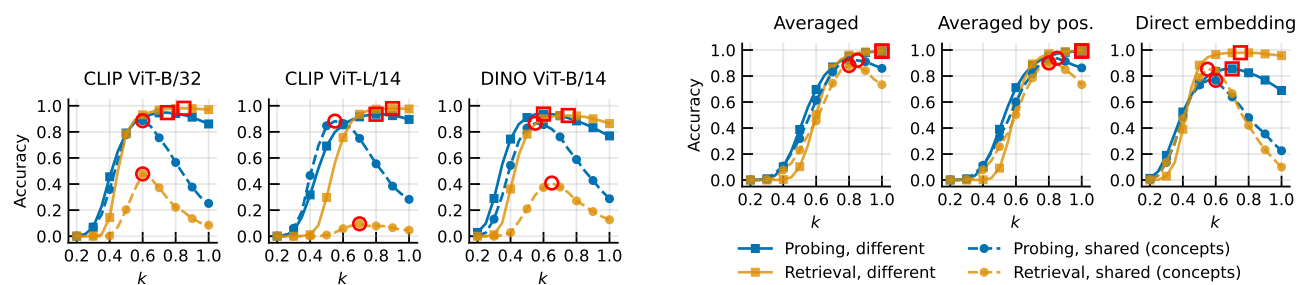


Figure 11. Effect of intervention strength on object replacement for CLEVR with embeddings obtained from single-object scenes.

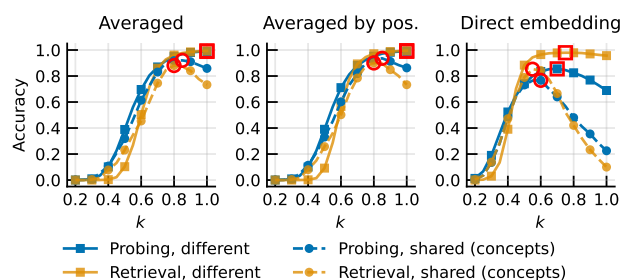


Figure 12. Effect of intervention strength on object replacement for text embeddings.